



**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERÍA**

**Escuela Profesional de Ingeniería de Sistemas**

**FinanceFlow: Plataforma Web de Gestión  
Financiera e Inteligencia Artificial para el Control  
de Gastos Personales**

Curso: Construcción de Software I

Docente: Mag. Ricardo Eduardo Valcárcel Alvarado

Integrantes:

***Sebastian Arce Bracamonte***

***(2019062886)***

***Brant Antony Chata Choque***

***(2020067577)***

**Tacna – Perú**

**2026**

# FinanceFlow: Plataforma Web de Gestión Financiera e Inteligencia Artificial para el Control de Gastos Personales

**Documento Informe de  
Factibilidad**

***Versión 1.0***

| CONTROL DE VERSIONES |           |              |              |            |                 |
|----------------------|-----------|--------------|--------------|------------|-----------------|
| Versión              | Hecha por | Revisada por | Aprobada por | Fecha      | Motivo          |
| 1.0                  | SAB       | SAB          | REVA         | 06/4/2026  | Primera Versión |
| 2.0                  | ACC       | ACC          | REVA         | 20/04/2026 | Segunda Versión |

|                                            |          |
|--------------------------------------------|----------|
| <b>Documento Informe de Factibilidad</b>   | <b>2</b> |
| 1. Descripción del Proyecto                | 5        |
| 1.1 Nombre del proyecto                    | 5        |
| 1.2 Duración del proyecto                  | 5        |
| 1.3 Descripción                            | 5        |
| 1.4 Objetivos                              | 5        |
| Objetivo general                           | 5        |
| Objetivos Específicos                      | 5        |
| 2. Riesgos                                 | 6        |
| 3. Análisis de la Situación actual         | 6        |
| 3.1 Planteamiento del problema             | 6        |
| 3.2 Consideraciones de hardware y software | 6        |
| 4. Estudio de Factibilidad                 | 7        |
| 4.1 Factibilidad Técnica                   | 7        |
| 4.2 Factibilidad Económica                 | 7        |
| 4.3 Factibilidad Operativa                 | 7        |
| 4.4 Factibilidad Legal                     | 8        |
| 4.5 Factibilidad Ambiental                 | 8        |
| 5. Conclusiones                            | 8        |

# 1. Descripción del Proyecto

## 1.1 Nombre del proyecto

*FinanceFlow: Plataforma Web de Gestión Financiera e Inteligencia Artificial para el Control de Gastos Personales*

## 1.2 Duración del proyecto

*Se estima una duración total de cuatro (4) meses para el desarrollo, pruebas e implementación del proyecto, distribuidos de la siguiente manera:*

| <i>Fase</i>                   | <i>Actividades Principales</i>                                                                                    | <i>Duración</i> |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------|
| Fase 1: Análisis y Diseño     | Levantamiento de requisitos, arquitectura del sistema, diseño de UI/UX, modelado de base de datos.                | 3 semanas       |
| Fase 2: Desarrollo Backend    | Implementación de la API RESTful (Node.js/Express), modelos MongoDB, autenticación JWT, endpoints CRUD.           | 4 semanas       |
| Fase 3: Desarrollo de IA      | Microservicio FastAPI, motor OCR (Tesseract), modelo Naive Bayes, endpoint /retrain para Silent Training.         | 3 semanas       |
| Fase 4: Desarrollo Frontend   | Interfaces React.js, integración con API, dashboards Recharts, diseño TailwindCSS responsive.                     | 4 semanas       |
| Fase 5: Integración y Pruebas | Pruebas unitarias, de integración, pruebas de carga, corrección de errores, revisión de seguridad.                | 2 semanas       |
| Fase 6: Despliegue y Entrega  | Despliegue en Vercel (Frontend), Render (Backend + ML Service), configuración MongoDB Atlas, documentación final. | 1 semana        |

## 1.3 Descripción

*FinanceFlow es una plataforma web de gestión financiera personal que utiliza Inteligencia Artificial, Reconocimiento Óptico de Caracteres (OCR) y Machine Learning para automatizar y optimizar el registro, categorización y análisis de los movimientos financieros de sus usuarios. El sistema permite registrar ingresos y gastos de forma manual o automática (a través de la fotografía de comprobantes de pago, yape o transferencias), visualizar el estado financiero mediante dashboards interactivos, recibir alertas ante gastos elevados y obtener categorías de gasto predichas inteligentemente por un modelo que se adapta continuamente a los hábitos del usuario.*

*La plataforma está construida sobre una arquitectura cliente-servidor desacoplada, compuesta por tres capas: un frontend web desarrollado con React.js y Tailwind CON, un backend principal (API RESTful) con Node.js, Express y MongoDB, y un microservicio de Inteligencia Artificial desarrollado en Python con FastAPI, que aloja tanto el motor OCR como el modelo de clasificación por Machine Learning.*

## 1.4 Objetivos

### Objetivo general

*Desarrollar e implementar una plataforma web de gestión financiera personal que integre tecnologías de Inteligencia Artificial — específicamente OCR y Machine Learning — para automatizar el registro, categorización y análisis de movimientos financieros, contribuyendo a mejorar los hábitos económicos y la educación financiera de sus usuarios.*

### Objetivos Específicos

- *Diseñar e implementar un módulo de lectura automática de comprobantes de pago mediante tecnología OCR capaz de extraer monto, descripción y fecha con una precisión superior al 85% en condiciones de iluminación adecuada.*
- *Desarrollar un microservicio de Inteligencia Artificial en Python/FastAPI que implemente un modelo de clasificación Naive Bayes para predecir la categoría de cada movimiento financiero registrado.*
- *Implementar un mecanismo de aprendizaje continuo que re-entrene el modelo de categorización cada vez que el usuario registra un nuevo movimiento, mejorando la personalización y precisión del sistema con el tiempo.*
- *Construir un dashboard analítico interactivo con Recharts que permita*

*visualizar el balance financiero, tendencias de gasto e ingresos y distribución por categorías en tiempo real.*

- *Desarrollar un sistema de alertas en tiempo real que notifique al usuario cuando un gasto registrado supere un umbral definido, excluyendo registros con borrado lógico.*
- *Implementar un sistema de autenticación seguro basado en JSON Web Tokens y un modelo de borrado lógico para la gestión de movimientos, garantizando la integridad y privacidad de los datos del usuario.*
- *Desplegar la plataforma en infraestructura cloud de alto rendimiento garantizando una disponibilidad mínima del 99% y tiempos de respuesta aceptables para el usuario final.*

## **2. Riesgos**

*A continuación se identifican los principales riesgos asociados al proyecto, junto con su probabilidad estimada, impacto y estrategia de mitigación:*

| <b>Riesgo</b>                                                       | <b>Probabilidad</b> | <b>Impacto</b> | <b>Estrategia de Mitigación</b>                                                                         |
|---------------------------------------------------------------------|---------------------|----------------|---------------------------------------------------------------------------------------------------------|
| Baja precisión del OCR con comprobantes de baja calidad fotográfica | Alta                | Alto           | Implementar validaciones de calidad de imagen y permitir corrección manual de datos extraídos.          |
| Degradación del modelo ML por datos de entrenamiento inconsistentes | Media               | Alto           | Implementar validaciones en el endpoint /retrain y mantener un conjunto de datos de entrenamiento base. |
| Limitaciones del plan gratuito de Render (cold start de 30-50 seg.) | Alta                | Medio          | Informar al usuario con un loader/estado de espera; considerar plan de pago en producción.              |

|                                                                           |       |          |                                                                                                        |
|---------------------------------------------------------------------------|-------|----------|--------------------------------------------------------------------------------------------------------|
| Vulnerabilidades de seguridad en datos financieros del usuario            | Baja  | Muy Alto | Implementar HTTPS, JWT con expiración corta, hashing de contraseñas (bcrypt) y validación de entradas. |
| Rotación del equipo de desarrollo durante el proyecto                     | Media | Alto     | Documentación técnica detallada y código comentado desde el inicio del proyecto.                       |
| Cambios en las APIs o dependencias de terceros (Tesseract, Scikit-Learn)  | Baja  | Medio    | Fijar versiones de dependencias y mantener un registro de cambios (changelog).                         |
| Sobrepasar los límites de almacenamiento de MongoDB Atlas (plan gratuito) | Media | Medio    | Implementar compresión de datos, borrado lógico y alertas de uso de almacenamiento.                    |

### **3. Análisis de la Situación actual**

#### **3.1 Planteamiento del problema**

*La gestión financiera personal efectiva es un desafío persistente para la mayoría de la población. En el contexto latinoamericano, donde los niveles de educación financiera formal son bajos y una parte significativa de las transacciones económicas se realizan en efectivo, las consecuencias de un control financiero deficiente son especialmente graves: endeudamiento innecesario, incapacidad de ahorro, estrés económico y falta de planificación futura.*

*Los métodos tradicionales de control de gastos presentan problemas estructurales bien documentados. El registro manual en hojas de cálculo o libretas requiere disciplina constante y es propenso a errores y omisiones. Los comprobantes de pago en papel se pierden o deterioran con facilidad, imposibilitando la reconciliación posterior de gastos. La mayoría de los usuarios no tiene acceso a herramientas de análisis financiero de calidad, o cuando las tienen, la curva de aprendizaje y complejidad las hace inaccesibles para el usuario no especializado.*



*Adicionalmente, las soluciones de software financiero existentes en el mercado presentan las siguientes limitaciones: (a) requieren entrada de datos completamente manual, generando fricción y abandono del sistema; (b) utilizan categorías de gasto estáticas que no se adaptan a los hábitos individuales del usuario; (c) no ofrecen mecanismos automáticos para digitalizar comprobantes físicos; y (d) carecen de inteligencia predictiva que ayude al usuario a entender sus patrones de consumo. FinanceFlow aborda cada una de estas deficiencias con soluciones tecnológicas concretas e innovadoras.*

### **3.2 Consideraciones de hardware y software**

El proyecto adopta un modelo de infraestructura totalmente basado en la nube (cloud-native), eliminando la necesidad de servidores propios y reduciendo los costos de operación iniciales. Las consideraciones técnicas son las siguientes:

| <b>Componente</b>           | <b>Tecnología / Servicio</b>                  | <b>Justificación</b>                                                                                           |
|-----------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Frontend (Hosting)          | Vercel (CDN global)                           | Despliegue automático desde repositorio Git, CDN global, soporte nativo para React.js, plan gratuito generoso. |
| Backend Principal (Hosting) | Render (Web Service)                          | Soporte para Node.js, despliegue automático, HTTPS automático, plan gratuito disponible para prototipo.        |
| Microservicio IA (Hosting)  | Render (Web Service Python)                   | Soporte para aplicaciones Python/FastAPI, permite instalar dependencias como Tesseract y Scikit-Learn.         |
| Base de Datos               | MongoDB Atlas (M0 Free Tier)                  | Base de datos NoSQL en la nube, 512MB gratuitos, alta disponibilidad, integración nativa con Mongoose.         |
| Acceso de Usuario           | Navegador Web moderno (Chrome, Firefox, Edge) | No requiere instalación de software adicional; compatible con dispositivos de escritorio y móviles.            |
| Cámara para OCR             | API MediaDevices del navegador (getUserMedia) | Acceso a la cámara del dispositivo del usuario directamente desde el navegador, sin plugins adicionales.       |

## **4. Estudio de Factibilidad**

### **4.1 Factibilidad Técnica**

*Desde el punto de vista técnico, el proyecto es completamente viable. Todas las tecnologías seleccionadas son maduras, ampliamente documentadas y cuentan con comunidades activas de soporte:*

- *React.js (v18+) y TailwindCSS son estándares de la industria para el desarrollo de interfaces web modernas y responsivas, con amplio soporte y ecosistema de componentes.*
- *Node.js con Express es una de las combinaciones más utilizadas para APIs RESTful, con excelente rendimiento para operaciones CRUD y soporte nativo para JSON.*
- *MongoDB con Mongoose ofrece flexibilidad en el esquema de datos, ideal para evolucionar el modelo de movimientos financieros, y MongoDB Atlas proporciona una solución gestionada en la nube.*
- *Python con FastAPI es la elección óptima para el microservicio de IA, dado que el ecosistema Python (Scikit-Learn, Tesseract-OCR) es el más maduro para aplicaciones de Machine Learning y procesamiento de imágenes.*
- *Tesseract OCR es el motor de reconocimiento de texto de código abierto más utilizado, con soporte para múltiples idiomas y capacidad de integración mediante la librería pytesseract.*
- *Scikit-Learn con Naive Bayes (MultinomialNB) es un algoritmo probado para clasificación de texto, especialmente efectivo para la categorización de descripciones de transacciones.*

*El equipo de desarrollo cuenta con las competencias técnicas necesarias en cada una de estas tecnologías, y la arquitectura de microservicios permite que cada componente sea desarrollado, probado y desplegado de manera independiente, reduciendo la complejidad de la integración.*

### **4.2 Factibilidad Económica**

*El proyecto presenta una factibilidad económica favorable, especialmente en su fase inicial de desarrollo y prototipado. La estrategia de infraestructura cloud con planes gratuitos mantiene los costos operativos en niveles mínimos durante el desarrollo y las primeras fases de uso:*

| Recurso                       | Costo Estimado (Mensual)    | Observaciones                                                                                        |
|-------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------|
| Vercel (Frontend Hosting)     | USD 0 (Plan Hobby)          | Hasta 100GB de ancho de banda; suficiente para prototipo y usuarios iniciales.                       |
| Render (Backend Node.js)      | USD 0 (Plan Free)           | 512MB RAM, 0.1 CPU; suficiente para carga moderada. Cold start de ~30 seg.                           |
| Render (Microservicio Python) | USD 0 (Plan Free)           | Mismas limitaciones que backend Node.js. Escalar a plan Starter (USD 7/mes) si la carga lo requiere. |
| MongoDB Atlas                 | USD 0 (M0 Free Tier)        | 512MB de almacenamiento. Para usuarios reales, migrar a M2 (USD 9/mes) o M5 (USD 25/mes).            |
| Dominio web personalizado     | USD 10–15 (Anual)           | Opcional para la versión de producción final.                                                        |
| Costos de Desarrollo          | Recursos propios del equipo | El equipo aporta su tiempo y conocimiento como inversión en el proyecto académico/emprendimiento.    |

*El costo total de operación en fase de prototipo es esencialmente cero en términos de infraestructura, lo que hace al proyecto altamente viable desde una perspectiva económica inicial. La escalabilidad está garantizada mediante la migración progresiva a planes de pago a medida que la base de usuarios crezca.*

### **4.3 Factibilidad Operativa**

*Operativamente, el sistema está diseñado para ser accesible sin entrenamiento previo especializado. La interfaz web intuitiva, el proceso de registro mediante fotografía y la retroalimentación inmediata del sistema reducen la curva de aprendizaje. Se identifican los siguientes factores que aseguran la viabilidad operativa:*

- La interfaz responsive permite acceder desde cualquier dispositivo con navegador web moderno, sin necesidad de instalación de aplicaciones adicionales.*
- El proceso de OCR proporciona retroalimentación visual inmediata, mostrando los datos extraídos antes de confirmar el registro, lo que permite correcciones rápidas.*

- *El sistema de categorización automática ahorra tiempo al usuario y reduce el esfuerzo cognitivo de clasificar manualmente cada transacción.*
- *Los dashboards visuales hacen que la información financiera sea comprensible para usuarios sin formación contable, fomentando la adopción continua de la plataforma.*
- *El sistema de alertas proactivo mantiene al usuario informado sin requerir que consulte activamente la plataforma.*

*Se recomienda elaborar una guía de usuario concisa (FAQ y tutoriales en video) para maximizar la adopción inicial de las funcionalidades más avanzadas como el OCR y el análisis de dashboards.*

## **4.4 Factibilidad Legal**

*La operación de una plataforma que almacena y procesa datos financieros personales conlleva responsabilidades legales significativas que han sido consideradas en el diseño del sistema:*

- *Protección de Datos Personales: El sistema implementa encriptación de contraseñas mediante bcrypt y transmisión de datos sobre HTTPS. Se recomienda elaborar una Política de Privacidad clara que cumpla con las regulaciones aplicables (Ley de Protección de Datos Personales del país de operación, GDPR si aplica para usuarios europeos).*
- *Almacenamiento de Datos Financieros: Los datos de transacciones son almacenados en MongoDB Atlas con controles de acceso basados en roles (RBAC). El modelo de borrado lógico garantiza que los datos no se eliminen físicamente de manera inmediata, cumpliendo con requisitos de auditoría.*
- *Propiedad Intelectual: Todas las dependencias de software utilizadas son de código abierto con licencias compatibles con uso comercial (MIT, Apache 2.0), incluyendo Tesseract OCR (Apache 2.0), Scikit-Learn (BSD), React (MIT) y FastAPI (MIT).*
- *Términos de Servicio: Se recomienda redactar unos Términos y Condiciones de Uso que delimiten la responsabilidad de la plataforma respecto a las decisiones financieras tomadas por el usuario basándose en la información proporcionada.*
- *Seguridad de autenticación: El uso de JWT con expiración configurable y la implementación de medidas antifuerza bruta garantizan el cumplimiento de buenas prácticas de seguridad reconocidas internacionalmente (OWASP).*

## 4.5 Factibilidad Ambiental

FinanceFlow presenta un perfil ambiental positivo en relación con las alternativas tradicionales de gestión financiera:

- **Reducción de papel:** Al digitalizar comprobantes de pago mediante OCR y promover el registro digital de transacciones, la plataforma contribuye directamente a la reducción del consumo de papel en el seguimiento financiero personal.
- **Infraestructura Cloud eficiente:** Los proveedores de infraestructura seleccionados (Vercel, Render, MongoDB Atlas) operan centros de datos con certificaciones de eficiencia energética y programas de carbono neutro, minimizando la huella ambiental del sistema.
- **Modelo de negocio digital:** Al ser una plataforma 100% web, no requiere distribución física de software ni hardware específico, eliminando la cadena logística asociada y su impacto ambiental.
- **Escalabilidad sin sobredimensionamiento:** La arquitectura de microservicios permite escalar únicamente los componentes que requieren más recursos en cada momento, optimizando el uso de energía en el servidor.

## 5. Análisis Financiero y de Costos

### 5.1 Costos de Software y Servicios Cloud (Costos Operativos Mensuales)

| Servicio / Herramienta           | Plan más barato pago | Qué incluye vs. plan gratuito                                                                                                    | Función en el Proyecto                                                                 | Costo Mensual |
|----------------------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|---------------|
| VercelPro                        | Pro — USD 20/mes     | Sin límite de builds, previews ilimitadas, analytics básico, soporte para dominios personalizados con protección por contraseña. | Hosting del Frontend React.js con mayor ancho de banda y sin restricciones de equipo.  | USD 20.00     |
| Render — Backend Node.js Starter | Starter — USD 7/mes  | 512 MB RAM, sin cold start (siempre activo), mayor CPU. Elimina la espera de 30–50 seg. del plan gratuito.                       | Hosting de la API RESTful (Node.js + Express) con disponibilidad continua garantizada. | USD 7.00      |

|                                            |                             |                                                                                                                                                          |                                                                                                                 |                                 |
|--------------------------------------------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|---------------------------------|
| Render —<br>Microservicio<br>PythonStarter | Starter — USD<br>7/mes      | Mismas mejoras que<br>el backend Node.js.<br>El microservicio de IA<br>requiere estar<br>siempre activo para<br>responder solicitudes<br>OCR sin demora. | Hosting del<br>microservicio<br>FastAPI (OCR<br>+ Naive<br>Bayes) sin<br>interrupciones<br>por<br>inactividad.  | USD<br>7.00                     |
| MongoDB<br>AtlasM2                         | M2 Shared —<br>USD 9/mes    | 2 GB de<br>almacenamiento (vs.<br>512 MB gratuito),<br>mejor rendimiento de<br>lecturas/escrituras,<br>backups automáticos<br>diarios.                   | Base de datos<br>NoSQL con<br>mayor<br>capacidad<br>para usuarios,<br>movimientos<br>e historial<br>financiero. | USD<br>9.00                     |
| GitHubTeam                                 | Team — USD<br>4/usuario/mes | Code owners,<br>revisiones de PR<br>obligatorias, más<br>minutos de GitHub<br>Actions para CI/CD<br>automático.                                          | Control de<br>versiones<br>colaborativo<br>con<br>protección de<br>ramas y<br>despliegue<br>automático.         | USD<br>4.00 (por<br>usuario)    |
| Dominio<br>WebAnual                        | Plan básico<br>.com / .app  | Dominio<br>personalizado con<br>privacidad WHOIS<br>incluida. Sin<br>diferencia funcional<br>vs. plan anual<br>gratuito de registro.                     | Dominio<br>personalizado<br>para la<br>plataforma en<br>producción<br>(ej.<br>financeflow.ap<br>p).             | USD<br>1.25<br>(~USD<br>15/año) |

|                                                                                        |                   |
|----------------------------------------------------------------------------------------|-------------------|
| <b>Total Costo Operativo Mensual (Planes Básicos Pagos —<br/>equipo de 3 personas)</b> | <b>USD 56.25</b>  |
| <b>Total Costo Operativo Anual</b>                                                     | <b>USD 675.00</b> |

### 5.3 Costos de Recursos Humanos (Estimación Fase de Desarrollo)

| Rol / Perfil                                                                                                                              | Responsabilidades Principales                                                 | Horas Estimadas | Tarifa/Hora (USD) | Costo Total  |
|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------|-------------------|--------------|
| Desarrollador Frontend<br>React.js · TailwindCSS · Recharts                                                                               | UI/UX, dashboards, integración con API, módulo de cámara web para OCR.        | 160 hrs         | USD 4.00          | USD 640.00   |
| Desarrollador Backend<br>Node.js · Express · MongoDB                                                                                      | API RESTful, autenticación JWT, CRUD de movimientos, borrado lógico, alertas. | 160 hrs         | USD 4.00          | USD 640.00   |
| Desarrollador IA / ML<br>Python · FastAPI · Scikit-Learn · Tesseract                                                                      | Microservicio OCR, modelo Naive Bayes, endpoint /retrain, Silent Training.    | 120 hrs         | USD 4.50          | USD 540.00   |
| QA / Tester<br>Pruebas funcionales e integración                                                                                          | Pruebas unitarias, integración, carga, validación de OCR y seguridad.         | 40 hrs          | USD 4.00          | USD 160.00   |
| Total Horas / Total Recursos Humanos                                                                                                      |                                                                               | 480 hrs         | —                 | USD 1,980.00 |
| * Tarifas estimadas para equipo académico / junior en mercado latinoamericano. En mercado freelance internacional: multiplicar x5–x8 para |                                                                               |                 |                   |              |

## 5.4 Resumen General de Presupuesto Inicial

| Categoría                  | Descripción                                                  | Frecuencia             | Costo (USD)            |
|----------------------------|--------------------------------------------------------------|------------------------|------------------------|
| Recursos Humanos           | Equipo de desarrollo: Frontend, Backend, IA/ML, QA (4 meses) | Única vez (desarrollo) | USD 1,980.00           |
| Hosting Frontend           | Vercel Hobby — React.js, CDN global                          | Gratuito               | USD 20.00              |
| Hosting Backend            | Render Free — Node.js API + Python Microservicio             | Gratuito               | USD 7.00               |
| Base de Datos              | MongoDB Atlas M0 — 512 MB NoSQL en la nube                   | Gratuito               | USD 9.00               |
| Dominio Web                | Registro de dominio .com o .app para producción              | Anual                  | USD 1.25 (~USD 15/año) |
| Herramientas de Desarrollo | GitHub, VS Code, Postman, Figma                              | Gratuito               | USD 4.00 (por usuario) |
| Contingencia (5%)          | Margen para imprevistos técnicos o ampliación de plazo       | Única vez              | USD 99.75              |

|                                                    |                 |
|----------------------------------------------------|-----------------|
| PRESUPUESTO TOTAL INICIAL ESTIMADO                 | USD 2,121       |
| Costo operativo mensual (fase prototipo)           | USD 1.25 / mes  |
| Costo operativo mensual (fase producción escalada) | USD 25–30 / mes |



## 6. Conclusiones

El análisis de factibilidad realizado en sus dimensiones técnica, económica, operativa, legal y ambiental concluye que el proyecto FinanceFlow es viable en todas sus dimensiones evaluadas.

Técnicamente, las tecnologías seleccionadas son maduras, gratuitas en su versión base y bien soportadas por la comunidad de desarrollo.

Económicamente, la estrategia de infraestructura cloud elimina los costos iniciales de hardware y mantiene los costos operativos mínimos durante la fase de prototipado.

Operativamente, la plataforma está diseñada para ser accesible por usuarios no especializados, con flujos de trabajo simplificados gracias a la automatización mediante OCR e IA.

Legalmente, el sistema incorpora las mejores prácticas de seguridad y privacidad de datos, con un marco legal claro mediante Política de Privacidad y Términos de Servicio.

Ambientalmente, la plataforma contribuye positivamente a la reducción del uso de papel y aprovecha infraestructura cloud con compromisos de sostenibilidad.